

Project Pantheon

The mini-dex thematic index construction pipeline.

Companion to docs/MINIDEX_SPEC.md

A non-technical guide to how the pipeline turns 8,000 SEC filings into 22 thematic stock baskets. Written for product managers and analysts who want the shape of the system without the code.

Contents

1	What this system does	1
2	The end-to-end journey of one company	2
3	Data-flow diagram	2
4	The role of each Python file	2
5	Major external dependencies	4
6	Hardware	4
7	Configuration knobs (config.json)	4
8	Why the multi-stage design	5
9	Exception rules and overrides	6
	Pre-revenue exception (Rule 5 of the scoring prompt)	6
	Anchor weight floor (Stage 7 index construction)	6
10	Outputs	6
	The three weight columns	6
11	The 22 buckets	7
	Silicon	7
	Interconnect & hardware	8
	Infrastructure	9
	Software & frontier	9
12	Downstream: prices and returns	10

1 What this system does

mini-dex classifies US-listed technology and AI-adjacent companies into 22 thematic sub-indices — “fabless chip design,” “AI-native data-centre power,” “frontier model labs,” and so on — and produces a weighted member list per theme that can be back-tested against real prices. Buckets are non-exclusive: NVIDIA earns weight in several themes at once, in proportion to how much

of its revenue plausibly comes from each. The output is a set of frozen CSV/Parquet files plus a manifest. It is a batch pipeline refreshed roughly annually, not a live service.

2 The end-to-end journey of one company

Follow NVIDIA (ticker NVDA) through the seven-stop pipeline.

1. Universe pull. Download the SEC’s master list of every US public filer — ~8,000 companies with tickers, names, and SIC industry codes. NVDA is one entry among banks, biotechs, oil producers, and everything else.
2. Industry filter. A curated SIC range keeps only tech-adjacent names (semis, computers, software, comms equipment, measuring instruments, telecom, data-centre REITs, electric utilities) — ~1,376 candidates. NVDA (SIC 3674) survives; JPMorgan and Pfizer do not.
3. Filing fetch and reading. Pull each candidate’s latest 10-K, the annual filing every US public company files with the SEC. Extract “Item 1: Business” — the plain-English section where the company describes what it sells — plus any structured revenue-by-segment data from XBRL (machine-readable tags inside the filing). For NVDA that gives both the narrative and a segment table (Data Center \$47.5B, Gaming \$10.4B, ...). Where XBRL is thin, an optional LLM step pulls dollar-quantified splits from the prose.
4. Embedding shortlist. Rather than score NVDA against all 22 buckets, the pipeline first uses embeddings — a technique that turns text into a list of numbers so similar topics land near each other — and cosine similarity, which measures how close two of those number lists are (0 = unrelated, 1 = identical topic). Only buckets above the 0.60 cut-off make the shortlist, typically 1–6 per company. NVDA lands on fabless chip design, AI accelerators, data-centre networking and a couple of neighbours — not foundry or frontier model labs.
5. LLM scoring. Shortlisted (company, bucket) pairs go to the Anthropic Message Batches API. The prompt gives the model NVDA’s Item 1 text, segment table, and just the shortlisted bucket definitions; it returns an estimated revenue share (0–1), a confidence label, and a one-line rationale per bucket. Every company is scored twice independently so QC can catch model disagreement.
- 6–7. QC and weighting. QC runs sanity checks — anchor companies must score highly on their buckets (NVDA above 0.5 on fabless chip design), the two runs must roughly agree, borderline scores get flagged. Build then converts scores into three parallel weights per bucket (equal, score, cap×score), each normalised to sum to 1. NVDA lands heavy in several silicon buckets and lighter in some software ones, and the whole run is frozen with a manifest.

3 Data-flow diagram

4 The role of each Python file

Each module owns one job and talks to the others only through the database or the file cache. That is the whole reason the pipeline can be resumed, retried, and re-run stage by stage without redoing the expensive parts.

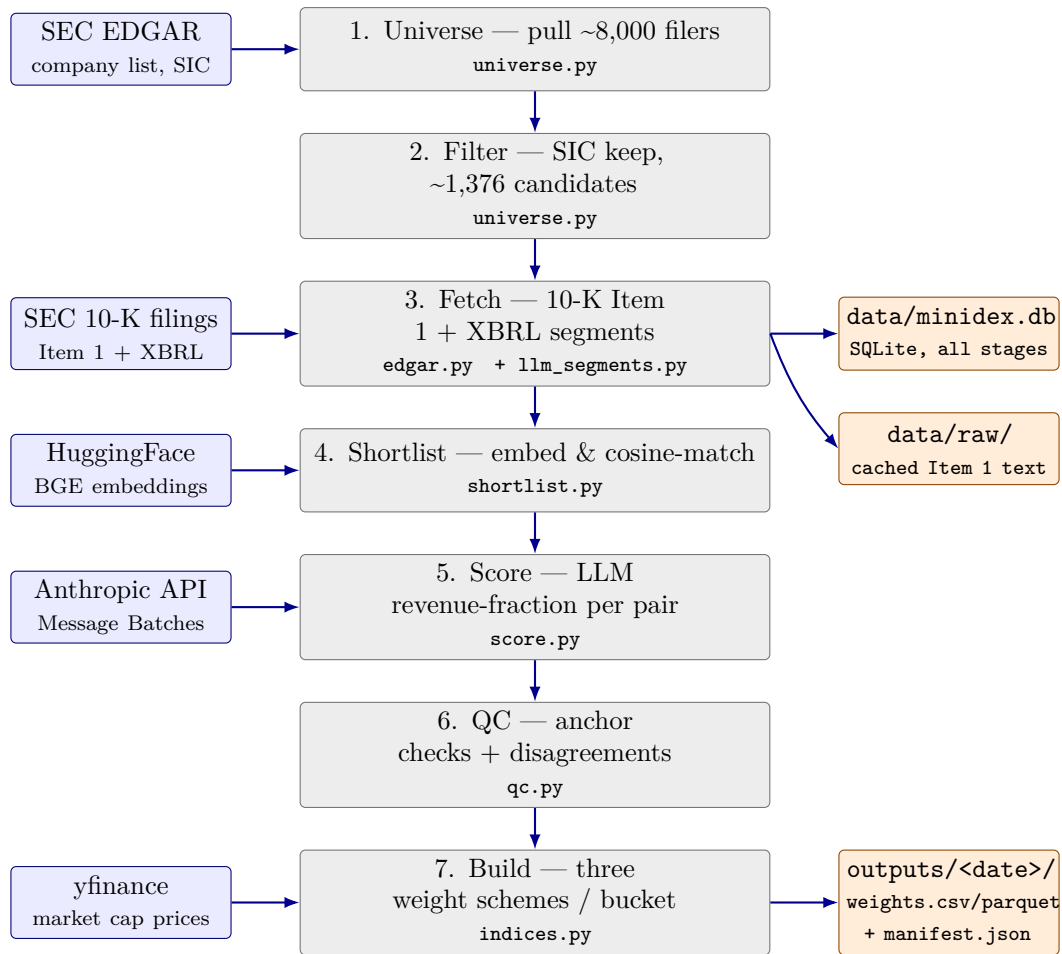


Figure 1: mini-dex data flow: external inputs (left) feed a seven-stage pipeline (centre) that writes to a SQLite DB, on-disk cache, and a frozen per-run output directory (right). Modules communicate only via the database and the file cache. Definitions of the 22 buckets live in `definitions/minindex_definitions.yaml`.

File	What it does, in one sentence
<code>config.py</code>	Loads API keys and paths from <code>.env</code> and exposes a single frozen settings object every other module reads.
<code>db.py</code>	Creates and opens the SQLite database and defines the tables (<code>companies</code> , <code>filings</code> , <code>shortlist</code> , <code>scores</code> , <code>batches</code>) plus the <code>latest_scores</code> view.
<code>universe.py</code>	Stages 1–2: pulls the SEC master company list, tags each row with an industry code, and marks the tech-adjacent slice as candidates.
<code>edgar.py</code>	Stage 3: for each candidate, downloads the latest 10-K, extracts the Item 1 business description, pulls segment revenue from the filing’s structured XBRL data, and records market cap.
<code>llm_segments.py</code>	Optional stage 3.5: for filings where XBRL segments are missing or useless, asks an LLM to extract quantified revenue splits from the 10-K text — refusing to guess if none are stated.
<code>shortlist.py</code>	Stage 4: turns each Item 1 and each bucket definition into embeddings and keeps only the (company, bucket) pairs whose cosine similarity clears the threshold.
<code>score.py</code>	Stage 5: builds the LLM prompt for every shortlisted pair, submits them via the Anthropic Message Batches API twice for independence, and parses the JSON responses into the <code>scores</code> table.
<code>models.py</code>	Pydantic schemas that the LLM output must match — a defensive layer that rejects malformed rows before they touch the

5 Major external dependencies

Service or library			What it's used for
SEC	EDGAR	(via edgartools)	The master list of US public filers, each company's SIC industry code, the latest 10-K annual report, and the machine-readable XBRL tags inside it that give segment revenue.
Anthropic	Message	Batches	The only paid input. Powers stage 5 scoring (revenue-fraction estimation per bucket) and the optional stage 3.5 segment extraction. Batches let us submit tens of thousands of prompts as one job at a discount.
HuggingFace		BAAI/bge-large-en-v1.5	Runs locally to turn text into embeddings for the stage 4 shortlist. Free, offline, deterministic. A smaller variant is available for faster iteration.
Yahoo Finance (via yfinance)			Recent share prices for computing market cap in stage 3, and optional price history for the stand-alone back-test script.
SQLite (stdlib)			A single local database file that stores everything each stage produces so the next stage can pick it up without re-running the previous one.

6 Hardware

The pipeline runs on a DigitalOcean droplet (cloud Linux server), not for speed but for availability: stage 3 fetches SEC filings for 6–8 hours at the SEC rate limit, and a laptop losing Wi-Fi or closing its lid mid-run would corrupt or delay it.

RAM is the constraining resource, driven almost entirely by stage 4 (embedding shortlist). The default model `BAAI/bge-large-en-v1.5` loads ~1.3 GB; PyTorch adds ~500 MB, OS overhead another ~500 MB, and encoder batches push the peak to 2.5–3 GB. 4 GB is the minimum that runs comfortably; the current 8 GB droplet has headroom.

CPU and disk are secondary. Two vCPUs roughly halve stage 4 wall time vs. one; more doesn't help much. Disk usage totals ~6 GB (venv, cached model, filings, DB) — the 160 GB the droplet ships with is vast overkill. If RAM is tight, swap `embedding_model` in `config.json` to `BAAI/bge-small-en-v1.5` (~130 MB; lower `similarity_threshold` to ~0.45 to compensate). Every other stage is I/O- or network-bound and runs in a few hundred MB.

7 Configuration knobs (`config.json`)

Six tuning parameters live in `config.json` at the repo root, hand-editable without touching code. Secrets (`ANTHROPIC_API_KEY`, `SEC_USER_AGENT`) stay in `.env`. Precedence: environment variable (`MINIDEX_<NAME>`) beats JSON beats hardcoded default — file-based defaults with per-run overrides.

Key	Default	What it controls
<code>embedding_model</code>	<code>BAAI/bge-large-en-v1.5</code>	HuggingFace sentence-transformer for the stage 4 shortlist. Default is 335 M parameters (~1.3 GB RAM). Swap to <code>BAAI/bge-small-en-v1.5</code> (~130 MB) for tight hosts — lower <code>similarity_threshold</code> too since small models produce lower cosine scores.
<code>similarity_threshold</code>	0.60	Cosine-similarity cutoff for stage 4. Pairs above go into the shortlist; anchors are always included regardless. Lower → more pairs scored → higher stage 5 cost, broader recall.
<code>score_floor</code>	0.10	Minimum LLM score required for a company to be included in a bucket’s final composition (stage 7). Rows below the floor are dropped as immaterial exposure. Anchor tickers can override the floor via <code>anchor_min_weight</code> .
<code>batch_model</code>	<code>claude-haiku-4-5</code>	Anthropic model powering stage 5 scoring and the optional stage 3.5 segment fallback. Haiku is fast and cheap; swap to Sonnet for higher-quality scoring at ~4× the cost.
<code>max_item1_chars</code>	12000	Truncation length for the 10-K Item 1 text sent to the LLM. ~12k chars is roughly 3k tokens — enough for most business descriptions, cheap enough for batch prompts.
<code>anchor_min_weight</code>	0.05	Minimum <code>weight_cap_score</code> for any anchor ticker in its bucket at stage 7. Below-floor anchors are force-included at exactly this weight; above-floor members scale down proportionally. Set 0.0 to disable and use pure <code>cap×score</code> .

8 Why the multi-stage design

A tempting shortcut would be to hand each company’s 10-K to an LLM once and ask “score this against all 22 buckets.” The pipeline is deliberately not built that way, and the biggest reason is cost.

1,376 candidates × 22 buckets would be ~30,000 LLM calls, each carrying a full bucket definition. The embedding shortlist collapses that: companies typically match 1–6 buckets, so the LLM sees ~22× fewer pairs. A full run scores ~1,376 companies for \$25–\$35; without the shortlist it would cost hundreds. Embeddings run locally for free — pushing as much filtering as possible into that stage is the biggest lever on budget.

Segment extraction is the second lever, on accuracy rather than cost. Showing the LLM “Data Center \$47.5B, Gaming \$10.4B” alongside the narrative anchors its revenue fractions in filed numbers rather than adjectives. When XBRL is silent, the optional LLM segment step recovers what it can from the prose without fabricating.

Third reason is iteration. Every stage persists its output, so cheap stages can be re-run (tweak SIC filter, similarity threshold, embedding model) without re-paying for scoring. Only stage 5 costs money, and it is gated behind a confirmation prompt with a cost estimate. QC and build are pure recomputation — freezing a new index from existing scores takes seconds.

9 Exception rules and overrides

Two overrides sit on top of the base “score = revenue fraction” rule to handle corner cases it cannot capture on its own.

Pre-revenue exception (Rule 5 of the scoring prompt)

A company with no material revenue but whose principal business purpose is the bucket’s defined activity should not score zero. SMR developers like Oklo and NuScale, quantum pure-plays like IonQ and D-Wave, early GPU cloud providers like Nebius — exactly the thematic exposure an index like this exists to capture. Rule 5 supersedes the revenue-fraction rule for these cases: pure-plays score 0.8–1.0, partial matches 0.3–0.7, aspirational-only names (established revenue base talking about future entry) stay at 0.0. Every pre-revenue score is flagged `pre_revenue: true` with confidence “low” for QC review.

Anchor weight floor (Stage 7 index construction)

Some legitimate anchors still score below the 0.10 `score_floor` that excludes immaterial exposures. Stage 7 guarantees representation via an anchor-weight floor: any anchor gets at least `anchor_min_weight` (default 5%) of its bucket’s `weight_cap_score`, even at score zero. Below-floor anchors are hard-fixed at the floor; above-floor members scale down proportionally so the bucket still sums to 1.0. If floors overrun 100%, they split 1.0 equally and everyone else drops to 0. Set `anchor_min_weight` to 0 in `config.json` to disable and use pure `cap × score` weighting.

These two overrides work in tandem. The pre-revenue exception fixes the scoring stage so that pure thematic plays get a real score. The anchor floor is the safety net at the weighting stage — if the LLM still misses a canonical name for any reason (definition ambiguity, run-to-run noise, or a genuinely borderline case), the anchor floor guarantees representation. In the pilot’s `power_generation` bucket, the exception moved Oklo and NuScale from 0.00 to 0.90, and the floor still guarantees them a 5% slot in the final index regardless.

10 Outputs

Every pipeline run writes a self-contained set of files to `outputs/<asof-date>/`:

File	What it is
<code>minindex_weights.csv</code>	Long-format table, one row per (<code>bucket_id</code> , <code>ticker</code>) member. Includes the LLM score, confidence, market cap, three weight columns (see below), both run rationales, and the prompt/model versions that produced it. The primary artefact for downstream back-testing.
<code>minindex_weights.parquet</code>	The same data as the CSV in a columnar binary format. Loads faster in pandas / DuckDB / Spark and preserves numeric precision.
<code>manifest.json</code>	Provenance: run date, as-of date, prompt version, model version, embedding model, thresholds, and SHA-256 hashes of the two source artefacts. Reproducibility guarantee.

The three weight columns

Every row carries three parallel weight values, all normalised so each bucket sums to 1.0. Pick one, blend them, or run all three through your back-test.

Column	Formula (per bucket)	What it captures
<code>weight_cap_score</code>	$(\text{market_cap} \times \text{score}) / \Sigma(\text{market_cap} \times \text{score})$	Finance-standard cap-weighting, adjusted for thematic exposure. Megacaps can dominate (NVDA hit ~52% of fabless_chip_design in the pilot). The only column the anchor-weight floor applies to.
<code>weight_score</code>	$\text{score} / \Sigma(\text{score})$	Ignores market cap. Weight is proportional to how much of the company's revenue is in the theme — AMD at score 1.0 and NVDA at 0.75 rival each other regardless of cap. Best for pure thematic exposure.
<code>weight_equal</code>	$1 / n$ (n = members after floor)	One share per member. Ignores cap and score magnitude. Naive but a useful diversification benchmark.

From the CSV alone you can derive further variants without re-running the pipeline — a max-weight cap, square-root or log cap-weighting, blended schemes, or confidence-filtered subsets. The heavy work is already done; iterating on weighting is a small pandas script.

11 The 22 buckets

Reader-friendly summaries of every bucket the pipeline scores against. Full definitions live in `definitions/minidex_definitions.yaml`. Grouped into the four families used throughout the pipeline.

Silicon

Bucket	Summary	Anchor tickers
Fabless chip design <code>fabless_chip_design</code>	Companies that design their own logic chips (GPUs, CPUs, SoCs, ASICs, DPUs, basebands, RF front-ends) and sell them under their own brand while outsourcing wafer fabrication. Revenue is chip sales, not IP licences or fab services.	NVDA, AMD, QCOM, MRVL, AVGO
Foundry <code>foundry</code>	Contract wafer manufacturers that fab chips other companies designed — process technology and advanced packaging sold as a service. The customer owns the design; the foundry sells fabrication capacity.	TSM, GFS, UMC
Semiconductor capital equipment <code>semicap_equipment</code>	Sellers of the tools used to fabricate, inspect, test and package chips — deposition, etch, lithography, metrology, inspection, ion implant and test systems — plus the associated service and spares revenue.	AMAT, LRCX, KLAC

Bucket	Summary	Anchor tickers
EDA & semiconductor IP eda_ip	Software for chip design (EDA suites) and licensable semiconductor IP such as processor cores, interface PHYs and verification tools. Revenue is licences, royalties and design services rather than physical chips.	SNPS, CDNS, ARM
Memory & storage memory_storage	Designers and/or manufacturers of memory and storage silicon and devices — DRAM, HBM, NAND flash, SSDs, hard drives, memory modules and controllers. Cloud storage services do not belong here.	MU, WDC, SNDK, STX
Microcontrollers & embedded microcontrollers_embedded	Vendors of MCUs, embedded processors, FPGAs and connectivity chips aimed at industrial, automotive, IoT and edge control workloads. This is general-purpose programmable silicon for embedded control, not data-centre compute.	MCHP, NXPI, STM
Power & analog semis analog_power	Analog, mixed-signal and power-management chips — amplifiers, data converters, voltage regulators and SiC/GaN power devices. Data-centre power delivery feeding high-wattage AI racks is a core in-scope activity.	TXN, ADI, MPWR, ON
Semiconductor materials & consumables semi_materials	Suppliers of the materials and chemistries consumed by fabs — specialty gases, CMP slurries, photoresists, filtration, wafer substrates, targets and related contamination-control kit. Distinct from the capital equipment used to apply them.	ENTG, MKSI, ACMR

Interconnect & hardware

Bucket	Summary	Anchor tickers
Photonics & optical interconnect photonics_optical	Optical and photonic components and modules — lasers, transceivers, silicon photonics, co-packaged optics and related test — used to move data inside and between data centres and across telecom networks.	COHR, LITE, AAOI
Networking silicon & systems networking	Data-centre and carrier networking gear: Ethernet switches and routers, switch/PHY/retimer/SerDes silicon, AI-cluster interconnect fabrics, DPUs/NICs and network operating software sold with the systems.	ANET, CRDO, ALAB, CIEN
Data centre hardware & servers dc_hardware	Designers and integrators of servers, GPU systems, rack-scale solutions and enterprise storage sold under their own brand. This is the system-level layer between chips and the facility, not contract manufacturing.	SMCI, DELL, HPE

Bucket	Summary	Anchor tickers
EMS / ODM manufacturing ems_odm	Contract assembly of servers, switches, boards and modules for OEM and hyperscaler customers, including design-and-build ODM arrangements. Distinguished from dc_hardware by the absence of an own-brand product line.	CLS, FLEX, JBL
Connectors & interconnect components connectors	Electrical connectors, cable assemblies, sockets, backplanes, busbars and thermal interface parts used in servers, networking gear and high-power electronics. Optical modules and semiconductors belong elsewhere.	APH, TEL

Infrastructure

Bucket	Summary	Anchor tickers
Hyperscalers hyperscalers	Operators of global, multi-region internet infrastructure. Qualifies via either public cloud (IaaS/PaaS sold to external customers) or self-operated hyperscale infrastructure powering the firm's own consumer/ad properties; a company can qualify on one leg or both.	MSFT, AMZN, GOOGL, ORCL, META
Neoclouds / GPU cloud neoclouds	Specialised AI clouds renting GPU capacity for training and inference, typically under contracts with AI labs and enterprises. Includes former crypto miners that have repurposed power and facilities into GPU hosting.	CRWV, NBIS, IREN, APLD
Data centre power & cooling equipment dc_power_cooling	The electrical and thermal facility layer inside data centres — UPS, switchgear, transformers, generators, liquid and air cooling, CDUs and related services that power and cool IT loads.	VRT, ETN, GEV
Power generation for data centres power_generation	Electricity producers where data-centre demand is a material and growing driver — independent power producers, nuclear operators and SMR developers with generation contracted to serve data-centre loads.	VST, CEG, TLN, OKLO, SMR
Colocation & data centre REITs colo_reits	Owners and lessors of data-centre real estate: space, power and interconnection sold to enterprises, hyperscalers and AI tenants, typically structured as REITs or wholesale facility operators.	EQIX, DLR

Software & frontier

Bucket	Summary	Anchor tickers
AI-native software & services <code>ai_native_software</code>	Software or services where AI/ML models are the core product value — AI platforms, model-driven analytics, speech and vision products, AI-driven diagnostics. Traditional SaaS that merely bolts on AI features scores low.	PLTR, AI, SOUN, TEM
Data infrastructure software <code>data_infrastructure</code>	The software layer that stores, moves, queries and monitors data at scale — cloud warehouses, lakehouses, operational databases, streaming, observability and pipeline tooling that underpins analytics and AI workloads.	SNOW, MDB, DDOG, CFLT
PQC & quantum <code>pqc_quantum</code>	Quantum computing hardware and cloud services, quantum networking, and post-quantum cryptography — quantum-resistant security chips, PQC software and migration services, QKD, and quantum processors offered via cloud access.	IONQ, RGTI, QBTS, LAES, ARQQ
Cybersecurity <code>cybersecurity</code>	Security software and services protecting enterprise and infrastructure environments — endpoint/EDR, network and cloud security, identity, SIEM/SOC and managed security — increasingly including AI-workload security. PQC-focused vendors sit in their own bucket.	CRWD, PANW, ZS, S

12 Downstream: prices and returns

The pipeline stops at frozen weights. Two standalone scripts sit downstream to close the loop from “these are the members and their weights” to “these are the returns.” They are outside the SQLite pipeline (no DB writes) and can be run independently against any historical `minindex_weights.csv`.

Script	What it does
<code>scripts/pull_prices.py</code>	Reads a weights CSV, extracts the unique ticker list, and fetches adjusted daily closes from Polygon.io with concurrent HTTP workers. Writes <code>data/prices.csv</code> . Handles rate limits and unknown-ticker responses. Requires <code>POLYGON_API_KEY</code> in <code>.env</code> .
<code>scripts/bucket_returns.py</code>	Reads the weights CSV and prices CSV, computes trailing 1y / 6m / 3m / 1m / 1w returns per bucket via $\sum(\text{weight_score} \times \text{ticker_return})$, with per-window renormalisation across members that priced. Windows are laid out longest-first to match how performance readouts are usually scanned. Writes a stylish standalone HTML report at <code>outputs/<asof>/bucket_returns.html</code> titled “Pantheon”, sorted by 1-year return. The report is interactive: each bucket row is clickable and expands to reveal a constituent table (ticker, full company name, weight, LLM score, market cap formatted as \$T/\$B/\$M, confidence) sorted by weight descending, with a chevron indicator that flips on expand. Company names are looked up at render time from <code>data/minindex.db</code> (<code>companies.name</code>); an optional <code>--db</code> flag overrides the path, and the report falls back to ticker-only if the DB is unavailable. Expand/collapse is pure inline JavaScript — no external dependencies.

Weight scheme is configurable via `--weight-col` (`weight_score` default; also `weight_cap_score` or `weight_equal`). Any of the three columns can be used to answer “how did this theme perform?” from different angles. This is where user iteration lives — the heavy lifting (SEC fetch, embedding shortlist, LLM scoring) already produced the CSV; deriving custom weighting schemes or return calculations is a small pandas exercise with no pipeline re-run required.

Generated for the mini-dex project. See *docs/MINIDEX_SPEC.md* for the full technical specification, *definitions/minindex_definitions.yaml* for the 22 bucket definitions, *prompts/scoring_prompt.md* for the current scoring rules, *docs/OUTPUT_COLUMNS.md* for the CSV column reference, and *README.md* for the run sequence.